

LAPORAN PRAKTIKUM STRUKTUR DATA

“Single Linked List”



OLEH:

LEXI MULIA YUNASPI

2511531006

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : DR WAHYUDI ,MT

ASISTEN PRAKTIKUM : M.GHALID

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis ucapkan kepada Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi dan pemahaman penulis terhadap materi praktikum yang telah dilakukan pada mata kuliah Struktur Data, khususnya mengenai penerapan Single Linked List dalam bahasa pemrograman Java.

Dalam praktikum ini, penulis mempelajari berbagai operasi pada struktur data Single Linked List seperti penambahan data, pencarian data, penghapusan data, traversal, serta implementasi studi kasus sistem antrian rumah sakit. Melalui praktikum ini, penulis memperoleh pemahaman tentang bagaimana struktur data digunakan untuk mengelola data secara dinamis dan efisien di dalam sebuah program.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan baik dari segi penulisan maupun isi pembahasan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik di masa mendatang. Semoga laporan ini dapat bermanfaat bagi pembaca dan menjadi referensi dalam mempelajari struktur data khususnya Single Linked List.

Padang, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat.....	2
BAB II PEMBAHASAN.....	3
2.1 Langkah Langkah Praktikum.....	3
2.2 Langkah Langkah Tugas Praktikum.....	45
BAB III PENUTUP.....	46
3.1 Kesimpulan.....	46
3.2 Saran.....	46
DAFTAR PUSTAKA.....	47

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep penting dalam ilmu komputer yang digunakan untuk mengatur dan menyimpan data agar dapat diproses secara efisien. Pemilihan struktur data yang tepat akan mempengaruhi performa program dalam melakukan operasi seperti penambahan, pencarian, penghapusan, dan penampilan data. Salah satu struktur data dinamis yang sering digunakan adalah Single Linked List. Single Linked List merupakan struktur data linear yang terdiri dari kumpulan node, di mana setiap node menyimpan data dan pointer yang menunjuk ke node berikutnya. Berbeda dengan array yang memiliki ukuran tetap, linked list memiliki ukuran yang fleksibel sehingga dapat bertambah atau berkurang sesuai kebutuhan program. Oleh karena itu, linked list banyak digunakan pada sistem yang membutuhkan pengelolaan data secara dinamis seperti sistem antrian, manajemen data, dan pengolahan data berurutan. Pada praktikum ini dilakukan implementasi berbagai operasi Single Linked List menggunakan bahasa pemrograman Java. Operasi yang dipelajari meliputi penambahan node di depan, belakang, dan posisi tertentu, pencarian data, penghapusan node, traversal linked list, serta penerapan linked list pada sistem antrian pasien rumah sakit. Praktikum ini bertujuan agar mahasiswa dapat memahami konsep dasar struktur data linked list serta mampu mengimplementasikannya dalam penyelesaian masalah pemrograman.

1.2 Tujuan

1. Memahami konsep dasar struktur data Single Linked List.
2. Mengimplementasikan operasi penambahan, pencarian, traversal, dan penghapusan node pada linked list menggunakan Java.
3. Memahami penggunaan pointer atau referensi antar node pada linked list.
4. Menerapkan struktur data linked list pada studi kasus sistem antrian rumah sakit.

5. Melatih kemampuan mahasiswa dalam membuat program berbasis struktur data secara terstruktur dan efisien.

1.3 Manfaat

1. Mahasiswa dapat memahami cara kerja struktur data Single Linked List.
2. Mahasiswa mampu membuat program Java menggunakan konsep linked list.
3. Mahasiswa dapat memahami proses pengelolaan data secara dinamis.
4. Mahasiswa memperoleh pengalaman dalam menerapkan struktur data pada kasus nyata seperti sistem antrian.
5. Mahasiswa dapat meningkatkan kemampuan logika pemrograman dan penyelesaian masalah menggunakan struktur data.

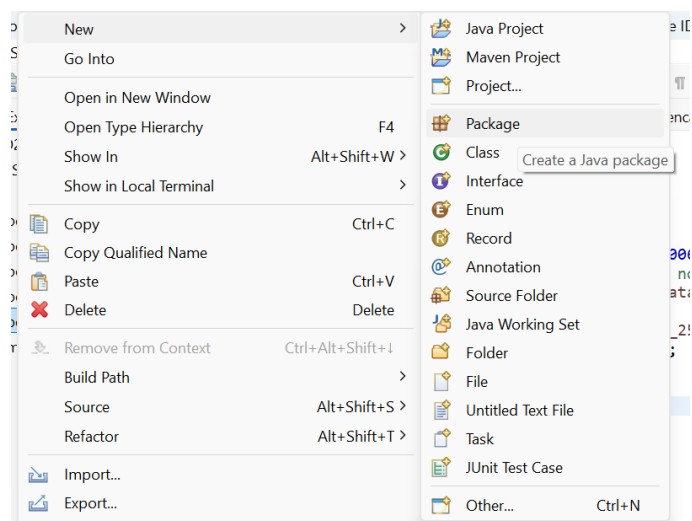
BAB II

PEMBAHASAN

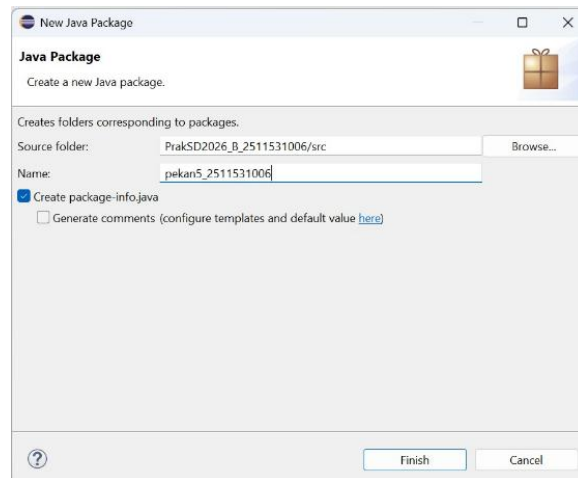
2.1 Laporan Praktikum

1. Program 1

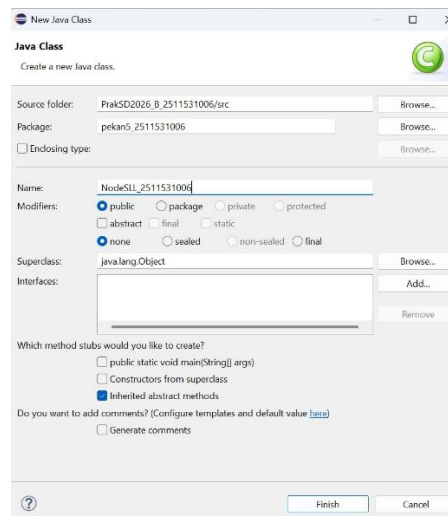
- a. Pada awal pembuatan program, dilakukan pembuatan package baru melalui menu New → Package pada Eclipse. Package digunakan untuk mengelompokkan file-file Java agar lebih rapi dan mudah diatur. Nama package yang dibuat adalah pekan5_2511531006. Dengan adanya package, semua class yang berkaitan dengan praktikum ini akan tersimpan dalam satu folder yang sama sehingga project menjadi lebih terstruktur dan mempermudah saat mencari maupun mengedit file program.



- b. Setelah package berhasil dibuat, langkah berikutnya adalah membuat class baru dengan nama NodeSLL_2511531006 melalui menu New → Class. Class ini digunakan sebagai node pada struktur data Single Linked List. Dalam linked list, node merupakan bagian utama yang berfungsi menyimpan data sekaligus menghubungkan node satu dengan node lainnya. Modifier public dipilih agar class dapat digunakan oleh class lain di dalam project.



- c. Pada bagian awal program terdapat deklarasi package yang menunjukkan bahwa class berada di dalam package pekan5_2511531006. Penulisan package penting agar Java mengetahui lokasi penyimpanan class serta membuat susunan program menjadi lebih rapi .



- d. Selanjutnya dibuat deklarasi class NodeSLL_2511531006. Class ini nantinya digunakan untuk membuat objek node pada Single Linked List. Tanda kurung kurawal pada class menunjukkan awal dan akhir isi class yang berisi atribut dan method.

```

1 package pekan5_2511531006;
2
3 public class NodeSLL_2511531006 {

```

- e. Di dalam class terdapat variabel `data_2511531006` yang digunakan untuk menyimpan nilai data pada node. Tipe data yang digunakan adalah integer sehingga data yang dapat disimpan berupa bilangan bulat. Variabel ini merupakan bagian utama dari node karena berfungsi sebagai tempat penyimpanan nilai yang dimasukkan pengguna.

```

4 // node bagian data
5 int data_2511531006;

```

- f. Selain variabel `data`, terdapat juga variabel `next_2511531006` yang berfungsi sebagai pointer atau penghubung ke node berikutnya. Variabel ini bertipe `NodeSLL_2511531006` karena akan menyimpan alamat node lain yang masih berada dalam class yang sama. Dalam Single Linked List, setiap node hanya memiliki satu pointer untuk menunjuk node setelahnya. Jika pointer bernilai null, maka node tersebut merupakan node terakhir dalam linked list.

```

6 // pointer ke node berikutnya
7 NodeSLL_2511531006 next_2511531006;

```

- g. Selanjutnya dibuat konstruktor class yang digunakan untuk memberikan nilai awal pada node saat objek dibuat. Konstruktor memiliki parameter `data_2511531006` yang berfungsi menerima nilai data yang akan dimasukkan ke dalam node. Konstruktor otomatis dijalankan ketika objek node dibuat sehingga node langsung siap digunakan.


```

8      // Konstruktor menginisialisasi node dengan data
9      public NodeSLL_2511531006(int data_2511531006)
10     {

```

- h. Di dalam konstruktor dilakukan proses pengisian data ke variabel data_2511531006. Kata this digunakan untuk membedakan variabel milik class dengan parameter konstruktor. Setelah itu pointer next_2511531006 diberi nilai null yang menandakan bahwa node belum terhubung dengan node lain.

```

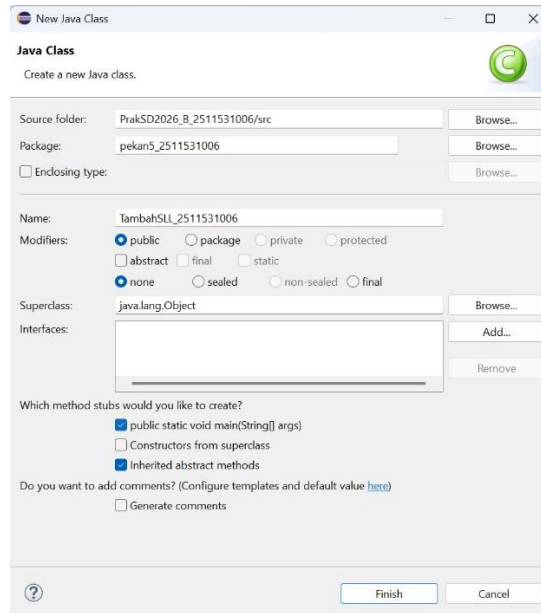
11         this.data_2511531006 = data_2511531006;
12         this.next_2511531006 = null;
13     }
14 }

```

- i. Secara keseluruhan, class NodeSLL_2511531006 digunakan sebagai dasar pembentukan struktur data Single Linked List karena memiliki dua bagian penting, yaitu data untuk menyimpan nilai dan pointer untuk menghubungkan node ke node berikutnya. Dengan adanya class ini, proses pembuatan linked list menjadi lebih mudah dan terstruktur.

2. Program 2

- a. Pada awal praktikum dibuat class baru bernama TambahSLL_2511531006 di dalam package pekan5_2511531006 melalui menu New → Class pada Eclipse. Class ini digunakan untuk membuat operasi penambahan data pada struktur data Single Linked List. Modifier public dipilih agar class dapat digunakan oleh class lain, sedangkan opsi public static void main(String[] args) dicentang agar program dapat langsung dijalankan. Dengan adanya class ini, semua operasi penambahan node dapat dituliskan dalam satu tempat.



- b. Pada bagian awal program terdapat deklarasi package `pekan5_2511531006` yang berfungsi untuk menunjukkan lokasi penyimpanan class dalam project Java. Setelah itu dibuat deklarasi class `TambahSLL_2511531006` yang menjadi tempat penulisan method-method operasi Single Linked List seperti menambah node di depan, di belakang, dan pada posisi tertentu. Penggunaan package membuat susunan file program menjadi lebih rapi dan mudah dipahami.

```

1 package pekan5_2511531006;
2
3 public class TambahSLL_2511531006 {
4

```

- c. Method `insertAtfront_2511531006` digunakan untuk menambahkan node di bagian depan Single Linked List. Method ini menerima parameter berupa head linked list dan nilai data yang akan dimasukkan. Method ini dibuat agar proses penambahan data di awal linked list menjadi lebih mudah dan cepat. Data yang dimasukkan nantinya akan berada pada posisi paling depan.

```

50 public static NodeSLL_2511531006 insertAtfront_2511531006(NodeSLL_2511531006 head_2511531006, int value_2511531006) {
6     NodeSLL_2511531006 new_node_2511531006 = new NodeSLL_2511531006(value_2511531006);
7     new_node_2511531006.next_2511531006 = head_2511531006;
8     return new_node_2511531006;
9 }

```

- d. Di dalam method `insertAtfront_2511531006`, program membuat node baru menggunakan class `NodeSLL_2511531006`. Setelah node dibuat, pointer `next_2511531006` pada node baru diarahkan ke head lama agar node sebelumnya tetap terhubung. Kemudian node baru dijadikan sebagai head baru linked list. Dengan cara ini susunan node lama tetap aman dan tidak hilang.

```

10 // fungsi menambah node di akhir SLL
11 public static NodeSLL_2511531006 insertAtEnd_2511531006(NodeSLL_2511531006 head_2511531006, int value_2511531006) {
12 {

```

- e. Method `insertAtEnd_2511531006` digunakan untuk menambahkan node di bagian akhir Single Linked List. Method ini menerima parameter head dan value sebagai data yang akan dimasukkan ke linked list. Dengan method ini, data baru dapat ditambahkan di posisi paling belakang. Method ini cocok digunakan saat ingin menambah data sesuai urutan input.

```

13 // buat sebuah node dengan sebuah nilai
14 NodeSLL_2511531006 newNode_2511531006= new NodeSLL_2511531006(value_2511531006);
15 // jika list kosong maka node jadi head
16 if (head_2511531006 == null) {
17     return newNode_2511531006;
18 }

```

- f. Pada method `insertAtEnd_2511531006`, program terlebih dahulu membuat node baru lalu memeriksa apakah linked list masih kosong atau tidak. Jika head bernilai null, maka node baru langsung dijadikan sebagai head karena belum ada node lain dalam linked list. Pengecekan ini penting agar program dapat berjalan tanpa error saat linked list kosong.

```

19 // simpan head ke variabel sementara
20 NodeSLL_2511531006 last_2511531006 = head_2511531006;
21 // telurusi ke node akhir
22 while (last_2511531006.next_2511531006 != null) {
23     last_2511531006 = last_2511531006.next_2511531006;
24 }

```

- g. Jika linked list sudah memiliki isi, program menggunakan variabel sementara last_2511531006 untuk menyimpan head sementara. Setelah itu dilakukan penelusuran node menggunakan perulangan while sampai menemukan node terakhir yang memiliki pointer next_2511531006 bernilai null. Variabel sementara digunakan agar head asli tidak berubah selama proses penelusuran.

```

25 // ubah pointer
26 last_2511531006.next_2511531006 = newNode_2511531006;
27 return head_2511531006;
28 }
29 }

```

- h. Setelah node terakhir ditemukan, pointer next_2511531006 pada node terakhir dihubungkan ke node baru sehingga node baru berhasil ditambahkan di bagian akhir linked list. Setelah proses selesai, method mengembalikan head linked list agar data tetap tersimpan. Dengan cara ini node-node tetap saling terhubung dengan baik.

```

30 static NodeSLL_2511531006 GetNode_2511531006(int data_2511531006) {
31     return new NodeSLL_2511531006(data_2511531006);
32 }
33

```

- i. Method GetNode_2511531006 digunakan untuk membuat dan mengembalikan node baru berdasarkan nilai data yang diberikan. Method ini membantu mempermudah proses pembuatan node sehingga kode program menjadi lebih rapi dan tidak perlu

ditulis berulang kali. Method ini juga membuat program lebih mudah dipelajari dan dikembangkan.

```
34 static NodeSLL_2511531006 insertPos_2511531006(NodeSLL_2511531006 headNode_2511531006, int position_2511531006, int value_2511531006)
35 NodeSLL_2511531006 head_2511531006 = headNode_2511531006;
36 if (position_2511531006 < 1)
37     System.out.print("invalid position");
38 if (position_2511531006 == 1) {
39     NodeSLL_2511531006 new_node_2511531006 = new NodeSLL_2511531006(value_2511531006);
40     new_node_2511531006.next_2511531006 = head_2511531006;
41     return new_node_2511531006;
42 } else {
```

- j. Method insertPos_2511531006 digunakan untuk menambahkan node pada posisi tertentu di Single Linked List. Method ini menerima parameter head node, posisi, dan nilai data yang akan dimasukkan. Program terlebih dahulu memeriksa apakah posisi yang dimasukkan valid atau tidak. Jika posisi kurang dari 1, maka program akan menampilkan pesan “invalid position”.

```
43 while (position_2511531006-- != 0) {
44     if (position_2511531006 == 1) {
45         NodeSLL_2511531006 newNode_2511531006 = GetNode_2511531006(value_2511531006);
46         newNode_2511531006.next_2511531006 = headNode_2511531006.next_2511531006;
47         headNode_2511531006.next_2511531006 = newNode_2511531006;
48         break;
49     }
50     headNode_2511531006 = headNode_2511531006.next_2511531006;
51 }
```

- k. Jika posisi valid, program akan menelusuri linked list menggunakan perulangan hingga mencapai posisi yang diinginkan. Setelah posisi ditemukan, program membuat node baru lalu mengatur pointer agar node baru terhubung dengan node sebelumnya dan node berikutnya. Dengan cara ini node baru dapat disisipkan tanpa merusak susunan linked list. Jika posisi melebihi jumlah node yang ada, program akan menampilkan pesan “Posisi di luar jangkauan”. Method ini membuat proses penambahan data menjadi lebih fleksibel sesuai kebutuhan pengguna.

```

52         if (position_2511531006 != 1)
53             System.out.print("Posisi di luar jangkuan");
54             return head_2511531006;
55     }
56 }

```

- j. Method printList_2511531006 digunakan untuk menampilkan seluruh isi data pada Single Linked List. Method ini menerima parameter berupa head_2511531006 yang merupakan node awal dari linked list. Fungsi utama method ini adalah untuk mencetak data dari node pertama hingga node terakhir secara berurutan sehingga isi linked list dapat dilihat oleh pengguna. Method ini sangat penting karena digunakan untuk menampilkan hasil dari setiap operasi penambahan node yang dilakukan pada linked list.

```

57 public static void printList_2511531006(NodeSLL_2511531006 head_2511531006) {
58     NodeSLL_2511531006 curr_2511531006 = head_2511531006;
59     while (curr_2511531006.next_2511531006 != null) {
60         System.out.print(curr_2511531006.data_2511531006 + "-->");
61         curr_2511531006 = curr_2511531006.next_2511531006;
62     }

```

- k. Di dalam method printList_2511531006, program membuat variabel sementara bernama curr_2511531006 yang diisi dengan head linked list. Variabel ini digunakan untuk membantu proses penelusuran node tanpa mengubah head asli linked list. Setelah itu program menggunakan perulangan while untuk bergerak dari satu node ke node berikutnya selama pointer next_2511531006 belum bernilai null. Pada setiap perulangan, data node dicetak menggunakan System.out.print() dengan tambahan tanda panah --> agar hubungan antar node terlihat jelas.

```

63         if (curr_2511531006.next_2511531006==null) {
64             System.out.print(curr_2511531006.data_2511531006);
65             System.out.println();
66         }
67     }

```

- l. Setelah proses perulangan selesai, program melakukan pengecekan apakah node saat ini merupakan node terakhir. Jika benar, maka data pada node terakhir dicetak tanpa tanda panah agar tampilan output menjadi lebih rapi. Setelah seluruh data selesai

dicetak, program menggunakan `System.out.println()` untuk membuat baris baru sehingga hasil output lebih mudah dibaca dan dipahami.

```
68 public static void main(String[] args) {  
69     // buat linked list 2->3->5->6  
70     NodeSLL_2511531006 head_2511531006 = new NodeSLL_2511531006(2);  
71     head_2511531006.next_2511531006 = new NodeSLL_2511531006(3);  
72     head_2511531006.next_2511531006.next_2511531006 = new NodeSLL_2511531006(5);  
73     head_2511531006.next_2511531006.next_2511531006.next_2511531006 = new NodeSLL_2511531006(6);
```

- m. Pada method main, program mulai membuat linked list awal dengan data 2->3->5->6. Pertama dibuat node head dengan nilai 2 menggunakan constructor `NodeSLL_2511531006`. Setelah itu setiap node dihubungkan menggunakan pointer `next_2511531006` hingga membentuk rangkaian linked list secara berurutan. Proses ini menunjukkan cara pembuatan Single Linked List secara manual menggunakan hubungan antar node.

```
74 // cetak list asli  
75 System.out.print("Senarai berantai awal:");  
76 printList_2511531006(head_2511531006);
```

- n. Setelah linked list awal selesai dibuat, program mencetak isi linked list menggunakan method `printList_2511531006`. Sebelum data ditampilkan, program mencetak tulisan “Senarai berantai awal:” sebagai penanda bahwa data yang ditampilkan merupakan linked list sebelum dilakukan perubahan atau penambahan node baru. Dengan langkah ini, pengguna dapat melihat kondisi awal linked list.

```
77 // tambahkan node baru di depan  
78 System.out.print("tambah 1 simpul di depan:");  
79 int data_2511531006 = 1;  
80 head_2511531006 = insertAtfront_2511531006(head_2511531006, data_2511531006);
```

- o. Selanjutnya program menambahkan node baru di bagian depan linked list menggunakan method `insertAtfront_2511531006`. Nilai yang dimasukkan adalah 1, sehingga node baru akan menjadi head baru linked list. Setelah proses selesai, susunan linked list berubah dari 2->3->5->6 menjadi 1->2->3->5->6. Program juga

menampilkan tulisan “tambah 1 simpul di depan:” sebagai informasi bahwa sedang dilakukan operasi penambahan node di awal linked list.

```
81 // tambahkan node baru di belakang
82 System.out.print("tambah 1 simpul di belakang:");
83 int data2_2511531006 = 7;
84 head_2511531006 = insertAtEnd_2511531006(head_2511531006, data2_2511531006);
--
```

- p. Berikutnya program menambahkan node baru di bagian akhir linked list menggunakan method `insertAtEnd_2511531006`. Nilai yang dimasukkan adalah 7, sehingga node baru ditempatkan pada posisi paling belakang linked list. Setelah proses selesai, linked list berubah menjadi 1->2->3->5->6->7. Program kemudian mencetak kembali isi linked list menggunakan method `printList_2511531006` agar hasil perubahan dapat dilihat secara langsung.

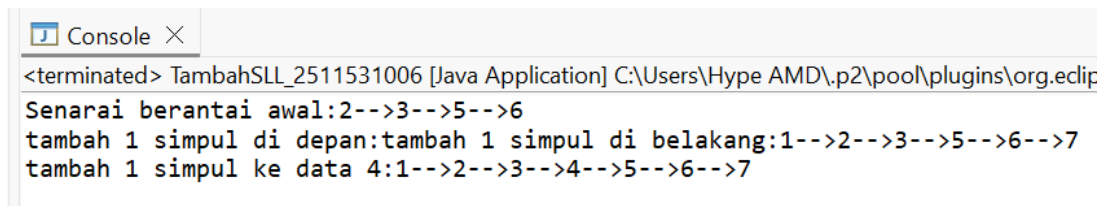
```
85 // cetak update list
86 printList_2511531006(head_2511531006);
87 System.out.print("tambah 1 simpul ke data 4:");
88 int data3_2511531006 = 4;
89 int pos_2511531006=4;
90 head_2511531006 = insertPos_2511531006(head_2511531006,pos_2511531006,data3_2511531006);
```

- q. Pada bagian akhir program dilakukan penambahan node pada posisi tertentu menggunakan method `insertPos_2511531006`. Nilai yang dimasukkan adalah 4 dan ditempatkan pada posisi ke-4. Program melakukan penelusuran node hingga posisi tujuan ditemukan, kemudian node baru disisipkan tanpa merusak hubungan antar node lainnya. Setelah proses selesai, program kembali mencetak isi linked list dan hasil akhirnya menjadi 1->2->3->4->5->6->7. Program ini menunjukkan bagaimana operasi penambahan node pada Single Linked List dapat dilakukan di depan, di belakang, maupun di posisi tertentu.

```
91 // cetak update list
92 printList_2511531006(head_2511531006);
93 }
94
95 }
```

- r. Pada tampilan console, program pertama kali menampilkan isi awal Single Linked List dengan tulisan "Senarai berantai awal" yaitu 2 -> 3 -> 5 -> 6. Setelah itu program menambahkan satu simpul di bagian depan menggunakan method `insert di depan` sehingga linked list berubah menjadi 1 -> 2 -> 3 -> 5 -> 6. Selanjutnya program menambahkan satu simpul di bagian belakang dengan nilai 7, sehingga hasil linked

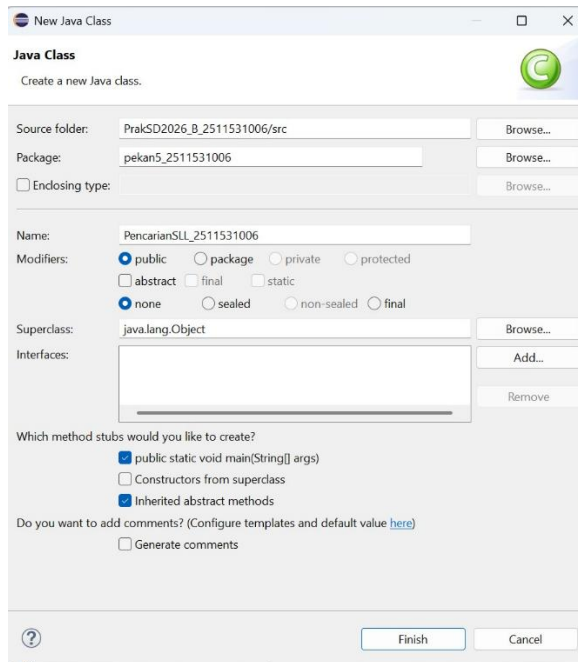
list menjadi 1 -> 2 -> 3 -> 5 -> 6 -> 7. Pada proses terakhir, program menambahkan node pada posisi tertentu yaitu data 4, sehingga susunan linked list berubah menjadi 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7. Output tersebut menunjukkan bahwa proses penambahan node di depan, belakang, dan posisi tertentu berhasil dilakukan pada Single Linked List.



```
<terminated> TambahSLL_2511531006 [Java Application] C:\Users\Hype AMD\p2\pool\plugins\org.eclipse
Senarai berantai awal:2-->3-->5-->6
tambah 1 simpul di depan:tambah 1 simpul di belakang:1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4:1-->2-->3-->4-->5-->6-->7
```

3. Program 3

- a. Pada tampilan **New Java Class**, terlihat proses pembuatan sebuah class Java baru di Eclipse. Class ini disimpan di dalam source folder PrakSD2026_B_2511531006/src dan berada pada package pekan5_2511531006. Penamaan package seperti ini biasanya menyesuaikan dengan minggu praktikum atau struktur tugas agar kode lebih rapi dan mudah dikelola. Nama class PencarianSLL_2511531006 menunjukkan bahwa program ini berkaitan dengan proses pencarian data pada **Single Linked List (SLL)** dan ditambahkan NIM sebagai identitas mahasiswa. Modifier yang dipilih adalah public, artinya class ini bisa diakses dari class lain. Selain itu, opsi public static void main(String[] args) dicentang supaya class ini bisa langsung dijalankan sebagai program utama.



- b. Pada potongan kode awal, terdapat deklarasi package `pekan5_2511531006`; yang berfungsi untuk menunjukkan bahwa class ini berada dalam package tersebut. Setelah itu dideklarasikan class `PencarianSLL_2511531006`. Class ini menjadi wadah utama untuk seluruh method yang berhubungan dengan pencarian dan penelusuran data pada Single Linked List. Struktur seperti ini sudah sesuai dengan standar Java, di mana satu file Java berisi satu class utama dengan nama yang sama seperti nama file.

```
1 package pekan5_2511531006;
2
3 public class PencarianSLL 2511531006 {
```

- c. Method static boolean `searchKey(NodeSLL_2511531006 head, int key)` digunakan untuk melakukan pencarian sebuah nilai tertentu (key) di dalam linked list. Parameter head merupakan node pertama dari linked list, sedangkan key adalah data yang ingin dicari. Di dalam method ini, dibuat variabel `curr` yang awalnya menunjuk ke head. Variabel ini berfungsi sebagai penunjuk (pointer) yang akan berjalan dari satu node

ke node berikutnya. Nantinya, method ini akan mengecek setiap data pada node apakah sama dengan key. Jika ditemukan, method akan mengembalikan nilai true, dan jika seluruh node sudah dicek tetapi tidak ada yang sama, maka akan mengembalikan false. Method ini bertipe boolean karena hasil pencariannya hanya ada dua kemungkinan: ditemukan atau tidak ditemukan.

```
4 static boolean searchKey(NodeSLL_2511531006 head, int key) {  
5     NodeSLL_2511531006 curr=head;
```

- d. Method public static void traversal(NodeSLL_2511531006 head) berfungsi untuk menelusuri dan menampilkan seluruh data yang ada di dalam linked list. Sama seperti method sebelumnya, parameter head menjadi titik awal penelusuran. Variabel curr diinisialisasi dengan head, lalu digunakan untuk berjalan ke setiap node sampai mencapai node terakhir. Proses traversal ini penting untuk melihat isi linked list secara keseluruhan, biasanya dengan mencetak data tiap node ke layar. Dengan adanya method traversal, pengguna program dapat memastikan data dalam linked list sudah tersimpan dan terhubung dengan benar sebelum atau sesudah dilakukan proses pencarian.

```
11 public static void traversal (NodeSLL_2511531006 head) {  
12     // mulai dari head  
13     NodeSLL_2511531006 curr = head;
```

- e. Pada proses penelusuran Single Linked List, program menggunakan perulangan while(curr != null) untuk membaca seluruh node dari awal sampai akhir. Variabel curr digunakan sebagai penunjuk node yang sedang diakses. Selama nilai curr belum null, data pada node akan ditampilkan menggunakan System.out.print. Setelah data dicetak, pointer dipindahkan ke node berikutnya dengan curr = curr.next_2511531006. Cara ini membuat proses traversal berjalan secara berurutan hingga mencapai node terakhir. Setelah seluruh data selesai ditampilkan, program

mencetak baris baru menggunakan `System.out.println()` agar tampilan output menjadi lebih rapi dan mudah dibaca.

```
14 // telusuri sampai pointer null
15 while(curr != null) {
16     System.out.print(" " + curr.data_2511531006);
17     curr = curr.next_2511531006; }
18 System.out.println(); }
```

- f. Pada method `main`, program mulai membuat Single Linked List dengan node pertama bernilai 4 yang disimpan pada variabel `head`. Setelah itu program menambahkan node berikutnya secara berurutan dengan nilai 14, 21, 30, dan 10. Setiap node dihubungkan menggunakan pointer `next_2511531006` sehingga membentuk rangkaian linked list yang saling terhubung dari node pertama hingga node terakhir. Proses ini menunjukkan cara pembuatan linked list secara manual menggunakan hubungan antar node.

```
19 public static void main(String[] args) {
20     NodeSLL_2511531006 head = new NodeSLL_2511531006(4);
21     head.next_2511531006 = new NodeSLL_2511531006(14);
22     head.next_2511531006.next_2511531006 = new NodeSLL_2511531006(21);
23     head.next_2511531006.next_2511531006.next_2511531006 = new NodeSLL_2511531006(30);
24     head.next_2511531006.next_2511531006.next_2511531006.next_2511531006 = new NodeSLL_2511531006(10);
}
```

- g. Setelah linked list berhasil dibuat, program menampilkan tulisan "Penelusuran SLL:" sebagai informasi bahwa isi linked list akan ditampilkan. Selanjutnya method `traversal(head)` dipanggil untuk melakukan penelusuran dan menampilkan seluruh isi data pada linked list. Hasil output akan memperlihatkan urutan data mulai dari node pertama hingga node terakhir sehingga pengguna dapat melihat isi linked list yang telah dibuat sebelumnya.

```
25 System.out.print("Penelusuran SLL:");
26 traversal(head);
```

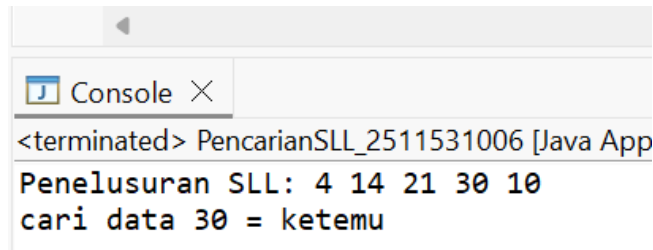
- h. Selanjutnya program menentukan data yang ingin dicari menggunakan variabel `key` yang diisi nilai 30. Program kemudian menampilkan tulisan "cari data 30 =" untuk memberikan informasi bahwa proses pencarian data sedang dilakukan. Setelah itu method `searchKey(head, key)` dipanggil untuk mengecek apakah nilai 30 terdapat di dalam linked list atau tidak. Method ini akan menelusuri node satu per satu sampai data ditemukan atau hingga mencapai akhir linked list.

```
27 // data yang kan dicari
28 int key = 30;
```

- i. Jika data yang dicari ditemukan, program akan menampilkan tulisan "ketemu" sebagai tanda bahwa nilai 30 berhasil ditemukan di dalam linked list. Namun jika data tidak ditemukan, program akan menampilkan tulisan "tidak ada". Proses ini menunjukkan cara kerja pencarian data pada Single Linked List dengan membandingkan nilai `key` terhadap data pada setiap node secara berurutan sampai hasil pencarian diperoleh.

```
29 System.out.print("cari data " + key+ " = ");
30 if (searchKey(head, key))
31     System.out.println("ketemu");
32 else
33     System.out.println("tidak ada");
34
35
36 }
37
38 }
```

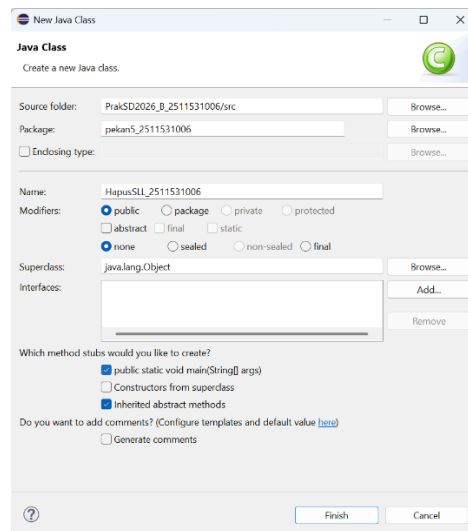
- j. Pada tampilan console, program menampilkan hasil penelusuran Single Linked List dengan tulisan "Penelusuran SLL" yang berisi data 4 14 21 30 10. Data tersebut ditampilkan secara berurutan dari node pertama hingga node terakhir menggunakan method `traversal`. Setelah itu program melakukan pencarian data dengan nilai 30 menggunakan method `searchKey`. Hasil output menunjukkan tulisan "cari data 30 = ketemu" yang berarti data 30 berhasil ditemukan di dalam Single Linked List.



```
<terminated> PencarianSLL_2511531006 [Java App]
Penelusuran SLL: 4 14 21 30 10
cari data 30 = ketemu
```

4. Program 4

- a. Pada tampilan New Java Class di Eclipse, program mulai membuat class Java baru dengan nama HapusSLL_2511531006. Class ini disimpan pada package pekan5_2511531006 yang berada di dalam source folder project. Modifier yang digunakan adalah public sehingga class dapat diakses dari class lain. Pada bagian method stub, program memilih public static void main(String[] args) agar class memiliki method utama untuk menjalankan program. Proses ini merupakan tahap awal pembuatan program operasi penghapusan data pada Single Linked List.



- b. Pada awal program, dituliskan deklarasi package pekan5_2511531006 sebagai penanda lokasi class berada. Setelah itu dibuat class HapusSLL_2511531006

menggunakan modifier public. Class ini digunakan untuk menyimpan seluruh method yang berhubungan dengan proses penghapusan node pada Single Linked List, seperti menghapus node awal, node akhir, dan node pada posisi tertentu.

```
1 package pekan5_2511531006;  
2  
3 public class HapusSLL_2511531006 {
```

- c. Program kemudian membuat method deleteHead dengan tipe data kembalian NodeSLL_2511531006. Method ini digunakan untuk menghapus node pertama atau head pada Single Linked List. Parameter yang digunakan adalah head yang menunjukkan node awal linked list. Setelah proses penghapusan selesai, method akan mengembalikan head terbaru sebagai awal linked list yang baru.

```
4 // fungsi untuk menghapus head  
5 public static NodeSLL_2511531006 deleteHead(NodeSLL_2511531006 head) {
```

- d. Pada proses penghapusan head, program terlebih dahulu melakukan pengecekan apakah linked list kosong menggunakan kondisi if(head == null). Jika kondisi tersebut benar, maka method langsung mengembalikan nilai null karena tidak ada node yang dapat dihapus. Pengecekan ini penting agar program tidak mengalami error saat proses penghapusan dilakukan pada linked list kosong.

```
7 if (head == null)  
8     return null;
```

- e. Jika linked list tidak kosong, program memindahkan posisi head ke node berikutnya menggunakan head = head.next_2511531006. Dengan cara ini, node pertama sebelumnya tidak lagi menjadi bagian dari linked list sehingga dianggap terhapus.

Setelah itu method mengembalikan head baru sebagai penanda awal linked list yang sudah diperbarui.

```
9 // pindahkan head ke node berikutnya
10 head = head.next_2511531006;
11 return head;}
12
```

- f. Selanjutnya program membuat method `removeLastNode` yang digunakan untuk menghapus node terakhir pada Single Linked List. Method ini menerima parameter `head` sebagai node awal linked list dan akan mengembalikan `head` setelah proses penghapusan selesai. Method ini digunakan agar operasi penghapusan node paling belakang dapat dilakukan secara otomatis melalui penelusuran linked list.

```
13 // fungsi menghapus node terakhir SLL
14 public static NodeSLL_2511531006 removeLastNode(NodeSLL_2511531006 head) {
```

- g. Pada method `removeLastNode`, program terlebih dahulu memeriksa apakah linked list kosong menggunakan kondisi `if(head == null)`. Jika linked list kosong, method langsung mengembalikan `null` karena tidak ada node yang bisa dihapus. Proses pengecekan ini dilakukan untuk menjaga agar program tetap aman saat dijalankan.

```
15 // jika list kosong , retur
16 if (head == null) {
17     return null;
18 }
```

- h. Setelah itu program memeriksa apakah linked list hanya memiliki satu node menggunakan kondisi `if(head.next_2511531006 == null)`. Jika kondisi tersebut benar, maka method akan mengembalikan `null` karena node satu-satunya dihapus sehingga

linked list menjadi kosong. Proses ini digunakan untuk menangani kasus khusus ketika jumlah node hanya satu.

```
19 // jika list satu node , hapus node dan return null
20 if (head.next_2511531006 == null) {
21     return null;
22 }
```

- i. Jika linked list memiliki lebih dari satu node, program membuat variabel secondLast yang diisi dengan head. Variabel ini digunakan untuk mencari node sebelum node terakhir. Program kemudian menggunakan perulangan while (secondLast.next_2511531006.next_2511531006 != null) untuk menelusuri linked list sampai menemukan node kedua terakhir. Pada setiap perulangan, pointer dipindahkan ke node berikutnya hingga posisi yang diinginkan ditemukan.

```
23 // temukan node terakhir ke dua
24 NodeSLL_2511531006 secondLast = head;
25 while (secondLast.next_2511531006.next_2511531006 != null) {
26     secondLast = secondLast.next_2511531006;
27 }
```

- j. Setelah node kedua terakhir ditemukan, program menghapus node terakhir dengan cara mengubah pointer secondLast.next_2511531006 = null. Dengan demikian hubungan menuju node terakhir diputus sehingga node terakhir tidak lagi menjadi bagian dari linked list. Setelah proses selesai, method mengembalikan head sebagai awal linked list yang sudah diperbarui.

```
28 // hapus node terakhir
29 secondLast.next_2511531006 = null;
30 return head; }
31
32
```

- k. Program juga membuat method deleteNode yang digunakan untuk menghapus node pada posisi tertentu di dalam Single Linked List. Method ini menerima parameter head dan position untuk menentukan node mana yang akan dihapus. Di dalam method dibuat variabel temp yang digunakan untuk penelusuran node serta variabel prev yang digunakan untuk menyimpan node sebelumnya. Method ini nantinya akan melakukan penelusuran linked list sampai posisi yang diminta ditemukan, kemudian node pada posisi tersebut dihapus tanpa merusak hubungan antar node lainnya.

```
33 // fungsi menghapus node di posisi tertentu
34 public static NodeSLL_2511531006 deleteNode(NodeSLL_2511531006 head, int position) {
35     NodeSLL_2511531006 temp = head;
36     NodeSLL_2511531006 prev = null;
```

- l. Pada bagian awal method deleteNode, program melakukan pengecekan apakah linked list kosong menggunakan kondisi if(temp == null). Jika kondisi tersebut benar, method langsung mengembalikan nilai head karena tidak ada node yang bisa dihapus. Pengecekan ini dilakukan agar program tidak mengalami error saat proses penghapusan dijalankan pada linked list yang belum memiliki data. Selain itu, proses ini menjaga agar program tetap berjalan dengan aman meskipun tidak ada isi linked list.

```
37 // jika linked list null
38 if (temp == null)
39     return head;
```

- m. Selanjutnya program menangani kondisi ketika node yang akan dihapus berada pada posisi pertama atau head menggunakan kondisi if(position == 1). Jika posisi yang diminta adalah posisi pertama, maka head dipindahkan ke node berikutnya menggunakan temp.next_2511531006. Dengan cara ini node pertama tidak lagi menjadi bagian dari linked list sehingga dianggap sudah terhapus. Setelah proses selesai, method mengembalikan head baru sebagai awal linked list yang terbaru.

```

40 // kasus 1: head dihapus
41 if (position == 1) {
42     head = temp.next_2511531006;
43     return head;}

```

- n. Jika node yang dihapus bukan node pertama, program melakukan penelusuran linked list menggunakan perulangan for. Variabel temp digunakan untuk menuju node yang akan dihapus, sedangkan variabel prev digunakan untuk menyimpan node sebelumnya. Pada setiap perulangan, pointer berpindah ke node berikutnya hingga mencapai posisi yang diminta. Proses ini bertujuan agar node dapat dihapus tanpa merusak hubungan antar node lainnya.

```

45 // kasus 2: menghapus node di tengah
46 // telusuri ke node yang di hapus
47 for (int i= 1; temp != null && i < position ; i++) {
48     prev = temp;
49     temp = temp.next_2511531006;}

```

- o. Setelah proses penelusuran selesai, program melakukan pengecekan menggunakan kondisi if(temp != null). Jika node ditemukan, maka node tersebut dihapus dengan cara menghubungkan node sebelumnya langsung ke node setelah node yang dihapus menggunakan prev.next_2511531006 = temp.next_2511531006. Namun jika node tidak ditemukan, program menampilkan pesan "Data tidak ada" sebagai informasi bahwa posisi yang diminta tidak tersedia di dalam linked list. Setelah itu method mengembalikan nilai head.

```

50 // jika ditemukan, hapus node
51 if (temp != null) {
52     prev.next_2511531006 = temp.next_2511531006;
53 } else {
54     System.out.println("Data tidak ada");}
55     return head;
56
57 }
58

```

- p. Pada method `printList`, program membuat variabel `curr` yang diisi dengan `head`. Variabel ini digunakan sebagai pointer sementara untuk membantu proses traversal linked list dari node pertama hingga node terakhir. Method ini berfungsi untuk menampilkan seluruh isi linked list secara berurutan sehingga pengguna dapat melihat data yang tersimpan di dalam linked list. Dengan adanya method ini, isi linked list dapat diperiksa dengan mudah setelah dilakukan operasi tertentu.

```
59 // fungsi mencetak SLL
60 public static void printList(NodeSLL_2511531006 head) {
61     NodeSLL_2511531006 curr = head;
62     while (curr.next_2511531006 != null) {
63         System.out.print(curr.data_2511531006+"-->");
64         curr = curr.next_2511531006;
65     }
66     if (curr.next_2511531006 == null) {
67         System.out.print(curr.data_2511531006);
68     }
69     System.out.println();
70 }
```

- q. Program kemudian menggunakan perulangan `while(curr.next_2511531006 != null)` untuk menelusuri linked list selama masih ada node berikutnya. Pada setiap perulangan, data node dicetak menggunakan `System.out.print(curr.data_2511531006 + "-->")` sehingga tampilan output terlihat saling terhubung menggunakan tanda panah. Setelah data ditampilkan, pointer `curr` dipindahkan ke node berikutnya agar proses traversal terus berjalan sampai mencapai node terakhir.

```
69 // kelas main
70
71 public static void main(String[] args) {
```

- r. Ketika traversal mencapai node terakhir, program melakukan pengecekan menggunakan kondisi `if(curr.next_2511531006 == null)`. Jika kondisi tersebut benar, maka data pada node terakhir dicetak tanpa tambahan tanda panah agar tampilan output menjadi lebih rapi. Setelah seluruh data selesai dicetak, program menampilkan baris baru menggunakan `System.out.println()`. Cara ini membuat hasil output linked list lebih mudah dibaca oleh pengguna.

```

72 // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
73 NodeSLL_2511531006 head = new NodeSLL_2511531006(1);
74 head.next_2511531006 = new NodeSLL_2511531006(2);
75 head.next_2511531006.next_2511531006 = new NodeSLL_2511531006(3);
76 head.next_2511531006.next_2511531006.next_2511531006 = new NodeSLL_2511531006(4);
77 head.next_2511531006.next_2511531006.next_2511531006.next_2511531006 = new NodeSLL_2511531006(5);
78 head.next_2511531006.next_2511531006.next_2511531006.next_2511531006.next_2511531006 = new NodeSLL_2511531006(6);

```

- s. Pada bagian main, program mulai menjalankan seluruh operasi Single Linked List. Method main digunakan sebagai pusat eksekusi program sehingga seluruh proses pembuatan linked list, penghapusan node, dan penampilan data dilakukan dari bagian ini. Semua method yang telah dibuat sebelumnya dipanggil dan dijalankan melalui method utama ini.

```

79 // cetak list awal
80 System.out.println("list awal:");
81 printList(head);

```

- t. Program kemudian membuat Single Linked List dengan susunan data 1 -> 2 -> 3 -> 4 -> 5 -> 6. Node pertama disimpan pada variabel head, kemudian setiap node berikutnya dihubungkan menggunakan pointer next_2511531006. Proses ini dilakukan secara manual dengan menghubungkan node satu per satu hingga terbentuk rangkaian linked list yang lengkap. Susunan data tersebut nantinya digunakan untuk proses penghapusan node pada beberapa posisi.

```

82 // hapus head
83 head = deleteHead(head);
84 System.out.print("List setelah head dihapus:");
85 printList(head);

```

- u. Setelah linked list selesai dibuat, program menampilkan tulisan "list awal:" lalu memanggil method printList(head) untuk mencetak seluruh isi linked list. Hasil output akan menampilkan seluruh data mulai dari node pertama hingga node terakhir

sesuai urutan penyimpanannya. Tampilan ini digunakan untuk memperlihatkan kondisi awal linked list sebelum dilakukan proses penghapusan data.

```
86 // hapus node terakhir
87 head = removeLastNode(head);
88 System.out.println("List setelah simpul terakhir di hapus:");
89 printList(head);
```

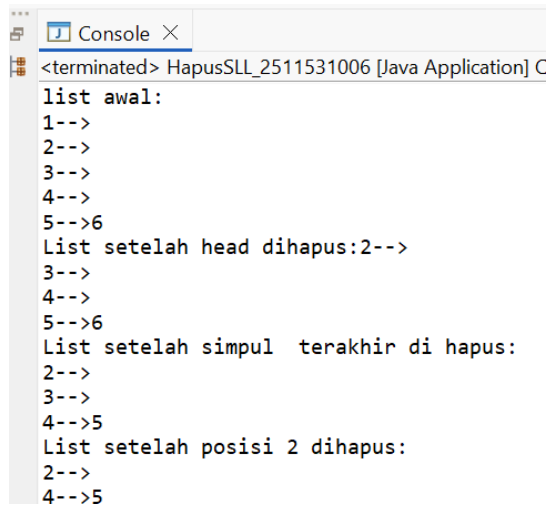
- v. Selanjutnya program melakukan penghapusan node pertama menggunakan method `deleteHead(head)`. Hasil penghapusan disimpan kembali pada variabel `head` karena node awal sudah berubah. Setelah proses selesai, program menampilkan tulisan "List setelah head dihapus:" kemudian mencetak kembali isi linked list menggunakan method `printList(head)` agar perubahan data dapat terlihat. Program kemudian melanjutkan proses dengan menghapus node terakhir menggunakan method `removeLastNode(head)` dan menampilkan hasil linked list terbaru setelah penghapusan dilakukan.

```
90 // Deleting node at position 2
91 int position =2;
92 head = deleteNode(head, position);
```

- w. Pada bagian akhir program, ditentukan posisi node yang akan dihapus menggunakan variabel `position` yang diisi nilai 2. Program kemudian memanggil method `deleteNode(head, position)` untuk menghapus node pada posisi tersebut. Setelah proses selesai, program menampilkan tulisan "List setelah posisi 2 dihapus:" lalu mencetak kembali isi linked list menggunakan method `printList(head)` sehingga hasil perubahan linked list dapat terlihat dengan jelas. Proses ini menunjukkan bahwa operasi penghapusan pada Single Linked List dapat dilakukan pada posisi tertentu sesuai kebutuhan pengguna.

```
93 // Print list after deletion
94 System.out.println("List setelah posisi 2 dihapus:");
95 printList(head);
96 }
97 }
```

- x. Pada tampilan console, program pertama kali menampilkan isi awal Single Linked List dengan tulisan "list awal:" yaitu 1 -> 2 -> 3 -> 4 -> 5 -> 6. Data tersebut ditampilkan menggunakan method `printList` yang melakukan traversal dari node pertama sampai node terakhir secara berurutan. Setelah itu program melakukan penghapusan node pertama menggunakan method `deleteHead(head)` sehingga linked list berubah menjadi 2 -> 3 -> 4 -> 5 -> 6. Perubahan ini terjadi karena node dengan nilai 1 sudah dihapus dan head berpindah ke node berikutnya. Selanjutnya program menghapus node terakhir menggunakan method `removeLastNode(head)`. Setelah proses selesai, console menampilkan linked list terbaru yaitu 2 -> 3 -> 4 -> 5, yang menunjukkan bahwa node terakhir dengan nilai 6 berhasil dihapus dari linked list. Program kemudian melanjutkan proses penghapusan node pada posisi tertentu menggunakan method `deleteNode(head, position)` dengan posisi 2. Setelah penghapusan dilakukan, linked list berubah menjadi 2 -> 4 -> 5 karena node pada posisi kedua yaitu data 3 berhasil dihapus. Output console tersebut menunjukkan bahwa seluruh operasi penghapusan pada Single Linked List berhasil dijalankan dengan benar mulai dari penghapusan head, node terakhir, hingga node pada posisi tertentu.

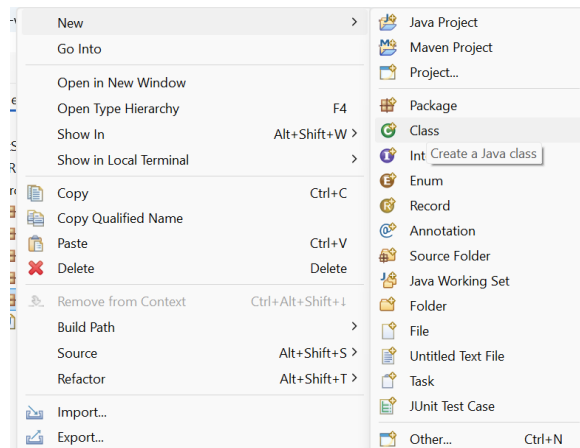


```
<terminated> HapusSLL_2511531006 [Java Application] C
list awal:
1-->
2-->
3-->
4-->
5-->6
List setelah head dihapus:2-->
3-->
4-->
5-->6
List setelah simpul terakhir di hapus:
2-->
3-->
4-->5
List setelah posisi 2 dihapus:
2-->
4-->5
```

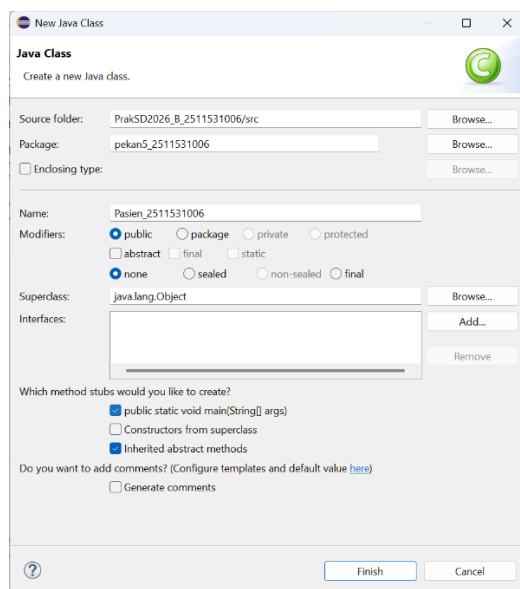
2.2 Langkah Langkah Tugas Praktikum

1. Program 1

- a. Pada tampilan Eclipse, program membuat class Java baru melalui menu **New** → **Class**. Class yang dibuat digunakan untuk menyimpan data pasien dalam bentuk node pada Single Linked List. Proses ini merupakan langkah awal pembuatan program antrian pasien menggunakan struktur data linked list. Dengan membuat class baru, program dapat memisahkan antara data pasien dan proses pengolahan data. Tampilan tersebut menunjukkan bahwa Eclipse menyediakan fitur untuk mempermudah pembuatan class Java secara otomatis.



- b. Gambar tersebut menunjukkan jendela **New Java Class** pada IDE Eclipse untuk membuat kelas Java baru. Source folder yang digunakan adalah `PrakSD2026_B_2511531006/src` dengan package `pekan5_2511531006`. Nama kelas yang dibuat yaitu `Pasien_2511531006` dengan modifier **public** agar dapat diakses dari package lain. Program menggunakan superclass default `java.lang.Object` dan opsi `public static void main(String[] args)` dicentang agar program bisa dijalankan. Setelah semua pengaturan selesai, tombol **Finish** digunakan untuk membuat kelas Java baru tersebut.



- c. Pada awal program dituliskan deklarasi package `pekan5_2511531006` sebagai penanda lokasi penyimpanan class di dalam project. Setelah itu dibuat class `Pasien_2411531006` menggunakan modifier `public` sehingga class dapat diakses dari class lain. Class ini digunakan untuk menyimpan seluruh atribut dan method yang berhubungan dengan data pasien. Pembuatan class ini menjadi dasar dalam pembentukan node pada Single Linked List. Nama class disesuaikan dengan kebutuhan program dan identitas praktikum mahasiswa.

```

1 package pekan5_2511531006;
2
3 public class Pasien_2411531006 {

```

- d. Program kemudian membuat beberapa atribut pada class pasien yaitu namaPasien_1006, penyakit_1006, nomorAntrian_1006, dan next_1006. Atribut namaPasien_1006 digunakan untuk menyimpan nama pasien, sedangkan penyakit_1006 digunakan untuk menyimpan jenis penyakit pasien. Atribut nomorAntrian_1006 bertipe integer dan digunakan untuk menyimpan nomor antrian pasien. Sementara itu atribut next_1006 digunakan sebagai pointer untuk menghubungkan node satu dengan node lainnya pada Single Linked List. Semua atribut dibuat menggunakan modifier private agar data lebih aman dan hanya bisa diakses melalui method tertentu.

```

4 // Atribut kelas Pasien
5 private String namaPasien_1006;
6 private String penyakit_1006;
7 private int nomorAntrian_1006;
8 private Pasien_2411531006 next_1006;
9

```

- e. Pada bagian constructor, program membuat constructor Pasien_2411531006 yang menerima parameter nama pasien, penyakit, dan nomor antrian. Constructor ini digunakan untuk menginisialisasi data pasien saat objek baru dibuat. Nilai parameter dimasukkan ke atribut class menggunakan keyword this sehingga data yang diterima langsung tersimpan pada objek. Selain itu, atribut next_1006 diisi dengan null karena node baru belum terhubung dengan node lainnya. Constructor ini mempermudah proses pembuatan objek pasien secara langsung dengan data lengkap.

```

10 // Constructor
11 public Pasien_2411531006(String namaPasien_1006, String penyakit_1006, int nomorAntrian_1006) {
12     this.namaPasien_1006 = namaPasien_1006;
13     this.penakit_1006 = penyakit_1006;
14     this.nomorAntrian_1006 = nomorAntrian_1006;
15     this.next_1006 = null;
16 }
17

```

- f. Program kemudian membuat method getter `getNamaPasien_1006()` yang berfungsi untuk mengambil atau menampilkan nilai atribut `namaPasien_1006`. Method ini mengembalikan nilai bertipe `String` berupa nama pasien yang tersimpan pada objek. Getter digunakan agar atribut `private` tetap bisa diakses dengan aman dari luar class. Dengan method ini, program dapat menampilkan nama pasien tanpa harus mengakses atribut secara langsung. Penggunaan getter juga membantu menjaga konsep enkapsulasi dalam pemrograman berorientasi objek.

```

18 // Getter (Selektor)
19 public String getNamaPasien_1006() {
20     return namaPasien_1006;
21 }
22

```

- g. Selanjutnya dibuat method getter `getPenyakit_1006()` yang digunakan untuk mengambil nilai atribut `penyakit_1006`. Method ini mengembalikan data bertipe `String` berupa penyakit yang diderita pasien. Getter ini memungkinkan data penyakit diakses dari class lain tanpa mengubah nilai aslinya. Cara ini membuat data pasien lebih terstruktur dan aman digunakan di dalam program. Method getter sangat penting ketika program ingin menampilkan informasi pasien pada proses antrian.

```

22
23 public String getPenyakit_1006() {
24     return penyakit_1006;
25 }
26

```

- h. Program juga membuat method getter `getNomorAntrian_1006()` yang digunakan untuk mengambil nilai atribut `nomorAntrian_1006`. Method ini mengembalikan nilai bertipe integer berupa nomor antrian pasien. Dengan adanya getter ini, nomor antrian dapat ditampilkan atau digunakan pada proses lain di dalam program. Penggunaan getter membantu menjaga keamanan data karena atribut tidak diakses secara langsung. Method ini biasanya digunakan saat proses penampilan data antrian pasien.

```
27 public int getNomorAntrian_1006() {  
28     return nomorAntrian_1006;  
29 }  
30
```

- i. Selain getter data pasien, program juga membuat method `getNext_1006()` yang digunakan untuk mengambil nilai pointer `next_1006`. Pointer ini berfungsi untuk menunjukkan node berikutnya pada Single Linked List. Method getter ini sangat penting karena digunakan dalam proses traversal atau penelusuran linked list. Dengan method ini, hubungan antar node dapat diakses dan dijalankan dengan lebih mudah. Penggunaan pointer menjadi bagian utama dalam struktur data linked list.

```
31 public Pasien_2411531006 getNext_1006() {  
32     return next_1006;  
33 }  
34
```

- j. Pada bagian setter, program membuat method `setNamaPasien_1006()` yang digunakan untuk mengubah nilai atribut `namaPasien_1006`. Method ini menerima parameter berupa nama pasien baru kemudian nilainya disimpan ke atribut menggunakan keyword `this`. Setter digunakan agar data private tetap dapat diperbarui dengan aman dari luar class. Dengan method ini, nama pasien dapat diubah sesuai kebutuhan program. Penggunaan setter juga mendukung konsep enkapsulasi dalam Java.

```

35 // Setter (Mutator)
36 public void setNamaPasien_1006(String namaPasien_1006) {
37     this.namaPasien_1006 = namaPasien_1006;
38 }
39

```

- k. Program kemudian membuat method `setPenyakit_1006()` yang digunakan untuk mengubah nilai atribut `penyakit_1006`. Method ini menerima parameter berupa penyakit baru lalu menyimpannya ke atribut class. Setter ini berguna ketika data penyakit pasien perlu diperbarui atau diperbaiki. Dengan cara ini, perubahan data tetap dilakukan melalui method sehingga lebih aman dibanding mengakses atribut secara langsung. Method setter membuat pengelolaan data pasien menjadi lebih terstruktur.

```

40 public void setPenyakit_1006(String penyakit_1006) {
41     this.penyakit_1006 = penyakit_1006;
42 }
43

```

- l. Selanjutnya dibuat method `setNomorAntrian_1006()` yang digunakan untuk mengubah nilai atribut `nomorAntrian_1006`. Method ini menerima parameter integer berupa nomor antrian baru kemudian menyimpannya ke atribut menggunakan keyword `this`. Setter ini memudahkan program ketika ingin memperbarui nomor antrian pasien. Dengan adanya method setter, data antrian dapat diatur dengan lebih fleksibel sesuai kebutuhan program. Penggunaan method ini juga membantu menjaga keamanan atribut `private`.

```

44 public void setNomorAntrian_1006(int nomorAntrian_1006) {
45     this.nomorAntrian_1006 = nomorAntrian_1006;
46 }
47

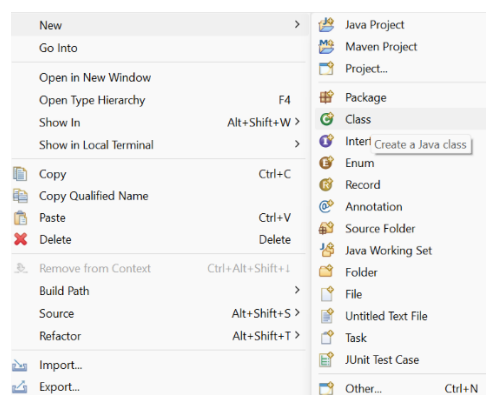
```

- m. Pada bagian terakhir, program membuat method `setNext_1006()` yang digunakan untuk menghubungkan node satu dengan node lainnya pada Single Linked List. Method ini menerima parameter berupa objek `Pasien_2411531006` yang akan dijadikan node berikutnya. Nilai parameter kemudian disimpan ke atribut `next_1006` menggunakan keyword `this`. Setter ini sangat penting dalam proses pembentukan linked list karena digunakan untuk mengatur hubungan antar node pasien. Dengan adanya method ini, struktur antrian pasien dapat dibentuk dan dijalankan dengan baik menggunakan Single Linked List.

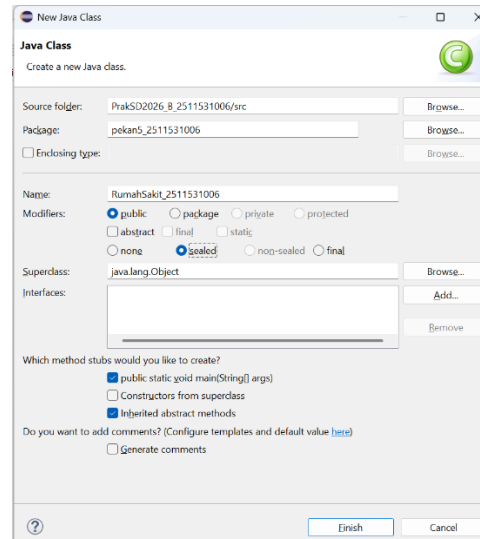
```
48 public void setNext_1006(Pasien_2411531006 next_1006) {  
49     this.next_1006 = next_1006;  
50 }  
51 }
```

2. Program 2

- a. Menu **New** → **Class** pada Eclipse digunakan untuk membuat class Java baru di dalam project. Langkah ini dilakukan sebelum menuliskan kode program agar file class tersedia di package yang dipilih. Pemilihan menu **Class** bertujuan untuk membuat program berbasis Object Oriented Programming (OOP). Class nantinya akan digunakan sebagai tempat penulisan atribut, method, dan logika program. Setelah dipilih, Eclipse akan menampilkan jendela konfigurasi pembuatan class Java.



- b. Jendela **New Java Class** digunakan untuk mengatur informasi class yang akan dibuat. Class diberi nama `RumahSakit_2511531006` dan ditempatkan pada package `pekan5_2511531006`. Modifier dipilih **public** agar class dapat diakses oleh class lain dalam project. Opsi `public static void main(String[] args)` dicentang supaya program dapat dijalankan langsung. Setelah semua pengaturan selesai, tombol **Finish** digunakan untuk membuat class baru.



- c. Bagian awal program berisi deklarasi package, import library, dan pembuatan class utama Java. Package digunakan untuk mengelompokkan file program agar lebih rapi dan mudah dikelola. Library `java.util.Scanner` diimport untuk menerima input data dari keyboard. Class utama bernama `RumahSakit_2511531006` yang berfungsi mengelola data pasien rumah sakit. Struktur ini merupakan dasar sebelum menambahkan atribut dan method pada program.

```
1 package pekan5_2511531006;
2
3 import java.util.Scanner;
4
5 public class RumahSakit_2511531006 {
```

- d. Variabel `head_1006` digunakan sebagai penunjuk node pertama pada linked list pasien. Tipe data `Pasien_2411531006` dipakai karena setiap node menyimpan data pasien. Variabel `counter_1006` berfungsi menghitung nomor antrian pasien secara otomatis. Nilai awal counter diatur 0 karena belum ada pasien yang terdaftar. Deklarasi variabel ini penting untuk membantu proses penyimpanan dan pengelolaan data pasien.

```
6      private Pasien_2411531006 head_1006;  
7      private int counter_1006 = 0;  
8
```

- e. Method `daftarkanPasien_1006()` digunakan untuk menambahkan data pasien baru ke dalam sistem. Program menggunakan objek `Scanner` agar pengguna dapat memasukkan data melalui keyboard. Input yang dimasukkan berupa nama pasien dan keluhan penyakit pasien. Data tersebut disimpan ke variabel `namaPasien_1006` dan `penyakit_1006`. Method ini menjadi proses utama dalam pendaftaran pasien rumah sakit.

```
9      // 1. Daftarkan Pasien (Insert at Tail)  
10     public void daftarkanPasien_1006() {  
11         Scanner sc = new Scanner(System.in);  
12         System.out.print("Masukkan Nama Pasien : ");  
13         String namaPasien_1006 = sc.nextLine();  
14         System.out.print("Masukkan Keluhan      : ");  
15         String penyakit_1006 = sc.nextLine();  
16     }
```

- f. Program menggunakan variabel `counter_1006` untuk membuat nomor antrian pasien secara otomatis. Nilai counter akan bertambah satu setiap kali ada pasien baru yang didaftarkan. Data pasien kemudian dibuat menjadi objek baru bernama `nodeBaru_1006`. Objek tersebut menyimpan nama pasien, keluhan, dan nomor antrian pasien. Dengan cara ini, setiap pasien mendapatkan nomor antrian sesuai urutan pendaftaran.


```

17 // Auto-increment nomor antrian
18 counter_1006++;
19 Pasien_2411531006 nodeBaru_1006 = new Pasien_2411531006(namaPasien_1006, penyakit_1006, counter_1006);
20

```

- g. Kondisi if (head_1006 == null) digunakan untuk mengecek apakah linked list masih kosong. Jika belum ada data pasien, maka node baru langsung dijadikan sebagai head linked list. Head berfungsi sebagai titik awal untuk mengakses seluruh data pasien dalam linked list. Langkah ini penting agar proses penyimpanan data pertama dapat berjalan dengan benar. Setelah head terisi, data pasien berikutnya dapat ditambahkan ke linked list.

```

21 // Jika list kosong, node baru langsung menjadi head
22 if (head_1006 == null) {
23     head_1006 = nodeBaru_1006;
24 } else {

```

- h. Variabel current_1006 digunakan untuk menelusuri node pasien dari awal hingga akhir linked list. Perulangan while berjalan selama node berikutnya masih tersedia atau belum bernilai null. Setelah node terakhir ditemukan, node pasien baru dihubungkan menggunakan setNext_1006(). Proses ini membuat data pasien tersimpan secara berurutan di dalam linked list. Program kemudian menampilkan pesan bahwa pasien berhasil didaftarkan beserta nomor antriannya.

```

25 // Menambahkan node pasien baru di akhir linked list
26 Pasien_2411531006 current_1006 = head_1006;
27 while (current_1006.getNext_1006() != null) {
28     current_1006 = current_1006.getNext_1006();
29 }
30 current_1006.setNext_1006(nodeBaru_1006);
31 }
32 System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + counter_1006 + "\n");
33 }
34
--

```

- i. Method panggilPasien_1006() digunakan untuk memanggil pasien dari antrian rumah sakit. Program terlebih dahulu mengecek apakah linked list masih kosong

menggunakan kondisi `head_1006 == null`. Jika antrian kosong, maka program akan menampilkan pesan bahwa tidak ada pasien yang bisa dipanggil. Perintah `return` digunakan untuk menghentikan method agar proses tidak dilanjutkan. Pengecekan ini penting untuk menghindari error saat mengambil data dari linked list kosong.

```
35 // 2. Panggil Pasien (Delete Head)
36 public void panggilPasien_1006() {
37     if (head_1006 == null) {
38         System.out.println("Antrian kosong! Tidak ada pasien yang bisa dipanggil.\n");
39         return;
40     }
41 }
```

- j. Program menampilkan data pasien yang sedang berada di posisi paling depan antrian. Data yang ditampilkan meliputi nomor antrian, nama pasien, dan keluhan penyakit pasien. Informasi tersebut diambil dari node `head_1006` menggunakan method `getter`. Bagian ini berfungsi untuk memberikan informasi pasien yang sedang dipanggil oleh rumah sakit. Output ditampilkan dengan format yang rapi agar mudah dibaca oleh pengguna.

```
42 // Menampilkan data pasien yang dipanggil
43 System.out.println("\n=== PASIEN DIPANGGIL ===");
44 System.out.println("Nomor Antrian: " + head_1006.getNomorAntrian_1006());
45 System.out.println("Nama Pasien : " + head_1006.getNamaPasien_1006());
46 System.out.println("Keluhan : " + head_1006.getPenyakit_1006());
47
```

- k. Setelah data pasien ditampilkan, posisi `head_1006` dipindahkan ke node berikutnya. Proses ini dilakukan menggunakan `head_1006 = head_1006.getNext_1006()`. Artinya pasien yang berada di depan antrian dianggap sudah selesai dipanggil dan keluar dari antrian. Konsep ini disebut operasi **Delete Head** pada struktur data linked list. Program kemudian menampilkan pesan bahwa pasien telah dipanggil dan keluar dari antrian.

```

48         // Head digeser ke node berikutnya
49         head_1006 = head_1006.getNext_1006();
50         System.out.println("Pasien telah dipanggil dan keluar dari antrian.\n");
51     }
52

```

1. Method tampilkanAntrian_1006() digunakan untuk menampilkan seluruh data antrian pasien. Program terlebih dahulu mengecek apakah linked list masih kosong atau tidak. Jika tidak ada data pasien, maka program akan menampilkan pesan “Antrian kosong!”. Perintah return digunakan untuk menghentikan method jika linked list kosong. Pengecekan ini dilakukan agar proses penelusuran data tidak mengalami error.

```

53         // 3. Tampilkan Antrian (Display)
54     public void tampilkanAntrian_1006() {
55         if (head_1006 == null) {
56             System.out.println("Antrian kosong!\n");
57             return;
58         }
59

```

- m. Program membuat variabel current_1006 untuk menelusuri linked list mulai dari head. Variabel posisi_1006 digunakan untuk menunjukkan urutan posisi pasien dalam antrian. Header “DAFTAR ANTRIAN PASIEN” ditampilkan agar output lebih jelas dan terstruktur. Node pertama pada linked list akan dijadikan titik awal penelusuran data pasien. Bagian ini mempersiapkan proses traversal untuk menampilkan seluruh isi linked list.

```

60         System.out.println("\n=== DAFTAR ANTRIAN PASIEN ===");
61         Pasien_2411531006 current_1006 = head_1006;
62         int posisi_1006 = 1;
63

```

- n. Perulangan `while(current_1006 != null)` digunakan untuk menelusuri semua node pasien. Program menampilkan posisi, nomor antrian, nama pasien, dan keluhan setiap pasien. Setelah satu node selesai ditampilkan, variabel `current_1006` berpindah ke node berikutnya. Variabel `posisi_1006` juga ditambah satu agar urutan pasien tetap sesuai. Proses traversal berhenti ketika node terakhir telah mencapai nilai `null`.

```
64      // Menelusuri linked list dari head hingga null
65      while (current_1006 != null) {
66          System.out.println("Posisi " + posisi_1006 + ":");
67          System.out.println(" No.Antri: " + current_1006.getNomorAntrian_1006());
68          System.out.println(" Nama      : " + current_1006.getNamaPasien_1006());
69          System.out.println(" Keluhan : " + current_1006.getPenyakit_1006());
70          System.out.println();
71          current_1006 = current_1006.getNext_1006();
72          posisi_1006++;
73      }
74  }
75  }
```

- o. Method `cariPasien_1006()` digunakan untuk mencari data pasien berdasarkan nama pasien. Program menggunakan objek `Scanner` untuk menerima input nama pasien dari keyboard. Input nama pasien diubah menjadi huruf kecil menggunakan `toLowerCase()` agar pencarian tidak sensitif huruf besar dan kecil. Variabel `current_1006` digunakan untuk menelusuri linked list pasien. Variabel boolean `ditemukan_1006` digunakan untuk menandai apakah data pasien ditemukan atau tidak.

```
76      // 4. Cari Pasien (Search) - CaseInsensitive
77      public void cariPasien_1006() {
78          Scanner sc = new Scanner(System.in);
79          System.out.print("Masukkan nama pasien yang dicari: ");
80          String namaCari_1006 = sc.nextLine().toLowerCase();
81
82          Pasien_2411531006 current_1006 = head_1006;
83          boolean ditemukan_1006 = false;
84      }
```

- p. Program melakukan pencarian pasien menggunakan perulangan `while` pada linked list. Setiap nama pasien dibandingkan dengan input pengguna menggunakan method

equals()).Jika data cocok, maka program menampilkan nomor antrian, nama, dan keluhan pasien tersebut.Variabel ditemukan_1006 diubah menjadi true lalu perulangan dihentikan menggunakan break.Jika tidak ada data yang cocok, maka program akan menampilkan pesan “Pasien tidak ditemukan”.

```

85 // Pencarian berdasarkan nama pasien secara CaseInsensitive
86 while (current_1006 != null) {
87     if (current_1006.getNamaPasien_1006().toLowerCase().equals(namaCari_1006)) {
88         System.out.println("\n== PASIEN DITEMUKAN ==");
89         System.out.println("No.Antri: " + current_1006.getNomorAntrian_1006());
90         System.out.println("Nama : " + current_1006.getNamaPasien_1006());
91         System.out.println("Keluhan : " + current_1006.getPenyakit_1006());
92         ditemukan_1006 = true;
93         break;
94     }
95     current_1006 = current_1006.getNext_1006();
96 }
97
98 if (!ditemukan_1006) {
99     System.out.println("Pasien tidak ditemukan!\n");
100 }
101 }
102

```

- q. Method cekStatusAntrian_1006() digunakan untuk melihat kondisi antrian pasien saat ini.Variabel jumlahPasien_1006 digunakan untuk menghitung total seluruh pasien dalam linked list.Variabel current_1006 digunakan untuk menelusuri data pasien mulai dari head.Method ini membantu pengguna mengetahui jumlahpasienyangmasihberadadalam antrian.Selain program juga dapat mengetahui siapa pasien yang berada paling depan.

```

103 // 5. Cek Status Antrian
104 public void cekStatusAntrian_1006() {
105     int jumlahPasien_1006 = 0;
106     Pasien_2411531006 current_1006 = head_1006;
107

```

- r. Program menggunakan perulangan while untuk menghitung jumlah node pasien dalam linked list.Setiap node yang dilewati akan menambah nilai jumlahPasien_1006.Jika jumlah pasien bernilai 0, maka program menampilkan pesan bahwa antrian kosong.Jika masih ada pasien, program akan menampilkan total pasien

beserta data pasien terdepan. Data pasien terdepan diambil dari node head_1006 menggunakan method getter.

```
108      // Menghitung jumlah total pasien
109      while (current_1006 != null) {
110          jumlahPasien_1006++;
111          current_1006 = current_1006.getNext_1006();
112      }
113
114      if (jumlahPasien_1006 == 0) {
115          System.out.println("Antrian kosong! Total pasien: 0\n");
116      } else {
117          System.out.println("Jumlah total pasien: " + jumlahPasien_1006);
118          System.out.println("Pasien terdepan:");
119          System.out.println("No.Antri: " + head_1006.getNomorAntrian_1006());
120          System.out.println("Nama      : " + head_1006.getNamaPasien_1006());
121          System.out.println("Keluhan : " + head_1006.getPenyakit_1006() + "\n");
122      }
123  }
124
```

- s. Method main() merupakan method utama yang digunakan untuk menjalankan program Java. Pada bagian ini dibuat objek rumahSakit_1006 dari class RumahSakit_2511531006. Program juga membuat objek Scanner untuk menerima input pilihan menu dari pengguna. Variabel pilihan_1006 digunakan untuk menyimpan menu yang dipilih pengguna. Bagian ini menjadi pusat utama jalannya seluruh fitur program rumah sakit.

```
125      public static void main(String[] args) {
126          RumahSakit_2511531006 rumahSakit_1006 = new RumahSakit_2511531006();
127          Scanner scanner_1006 = new Scanner(System.in);
128          int pilihan_1006;
129
```

- t. Program menggunakan perulangan do-while untuk menampilkan menu secara berulang. Menu yang tersedia meliputi daftar pasien, panggil pasien, tampilkan antrian, cari pasien, cek status antrian, dan keluar program. Pengguna diminta memasukkan pilihan menu sesuai fitur yang ingin digunakan. Input pilihan disimpan ke variabel pilihan_1006 menggunakan method nextInt(). Perulangan akan terus berjalan sampai pengguna memilih menu keluar.

```

130 do {
131     System.out.println("=== Antrian Rumah Sakit NIM: 2411531006===");
132     System.out.println("1. Daftarkan Pasien (Insert)");
133     System.out.println("2. Panggil Pasien (Delete Head)");
134     System.out.println("3. Tampilkan Antrian (Display)");
135     System.out.println("4. Cari Pasien (Search)");
136     System.out.println("5. Cek Status Antrian");
137     System.out.println("6. Keluar");
138     System.out.print("Pilihan: ");
139
140     pilihan_1006 = scanner_1006.nextInt();
141     scanner_1006.nextLine();
142

```

- u. Struktur switch-case digunakan untuk menjalankan fitur sesuai pilihan menu pengguna. Setiap case memanggil method tertentu seperti daftar pasien, panggil pasien, atau cari pasien. Jika pengguna memilih angka 6, maka program akan menampilkan pesan terima kasih dan berhenti. Bagian default digunakan untuk menampilkan pesan jika input menu tidak valid. Perulangan do-while berhenti ketika nilai pilihan_1006 sama dengan 6.

```

143 switch (pilihan_1006) {
144     case 1:
145         rumahSakit_1006.daftarkanPasien_1006();
146         break;
147     case 2:
148         rumahSakit_1006.panggilPasien_1006();
149         break;
150     case 3:
151         rumahSakit_1006.tampilkanAntrian_1006();
152         break;
153     case 4:
154         rumahSakit_1006.cariPasien_1006();
155         break;
156     case 5:
157         rumahSakit_1006.cekStatusAntrian_1006();
158         break;
159     case 6:
160         System.out.println("Terima kasih!");
161         break;
162     default:
163         System.out.println("Pilihan tidak valid!\n");
164 }
165 } while (pilihan_1006 != 6);
166 }
167 }

```

- v. Tampilan console menunjukkan hasil eksekusi program sistem antrian rumah sakit berbasis Java. Program menampilkan beberapa menu utama seperti daftar pasien, panggil pasien, tampilkan antrian, cari pasien, cek status antrian, dan keluar. Pengguna memilih menu 1 untuk melakukan proses pendaftaran pasien baru ke dalam antrian. Setelah itu, pengguna memasukkan nama pasien Dina dan keluhan penyakit

Demam melalui keyboard. Program kemudian berhasil menyimpan data pasien dan memberikan nomor antrian otomatis yaitu nomor 1.

```
Console X
RumahSakit_2511531006 [Java Application] C:\Users\Hype AMD\p2\
=== Antrian Rumah Sakit NIM: 2411531006===
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : Dina
Masukkan Keluhan : Demam
Pasien berhasil didaftarkan! Nomor Antrian: 1
```


BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Single Linked List merupakan struktur data dinamis yang sangat berguna dalam pengelolaan data secara fleksibel. Melalui praktikum ini, mahasiswa dapat memahami bagaimana proses penyimpanan data dilakukan menggunakan node yang saling terhubung melalui pointer. Berbagai operasi seperti penambahan node di depan, belakang, dan posisi tertentu, pencarian data, traversal, serta penghapusan node berhasil diimplementasikan menggunakan bahasa pemrograman Java. Selain itu, penerapan Single Linked List pada sistem antrian rumah sakit menunjukkan bahwa struktur data ini dapat digunakan dalam penyelesaian masalah nyata yang membutuhkan pengelolaan data secara berurutan. Program yang dibuat mampu melakukan proses pendaftaran pasien, pemanggilan pasien, pencarian data pasien, serta pengecekan status antrian dengan baik. Praktikum ini juga membantu mahasiswa memahami pentingnya penggunaan struktur data yang tepat untuk meningkatkan efisiensi dan kerapian program.

3.2 Saran

Dalam pelaksanaan praktikum selanjutnya, diharapkan mahasiswa dapat lebih memahami konsep dasar pointer dan hubungan antar node pada linked list agar tidak mengalami kesulitan saat membuat program. Selain itu, mahasiswa juga disarankan untuk lebih banyak berlatih membuat berbagai variasi program struktur data agar kemampuan logika pemrograman semakin meningkat. Pengembangan program juga dapat dilakukan dengan menambahkan fitur yang lebih kompleks seperti penggunaan Double Linked List, Circular Linked List, atau penyimpanan data menggunakan database. Dengan adanya pengembangan tersebut, pemahaman mahasiswa mengenai struktur data dan implementasinya dalam dunia nyata akan menjadi lebih baik.

DAFTAR PUSTAKA

Data Structures and Algorithms in Java, Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java*. John Wiley & Sons.